Project - Low Level Design
On
Project Title: Chat with Logistics Documents (RAG)

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
	SUHANI MOGRA	EN22CS301989
	SNEHA ARAS	EN22CS301964
	NAMAN PATIDAR	EN22ME303013
	VAIBHAV SHRIVASTAVA	EN22CS3011051

Group Name: *Group 05D9*

Project Number:05

Industry Mentor Name:

University Mentor Name: Vineeta Rathore

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. UML daigrams

4. References and Conclusion

1. Introduction

The Logistics AI Assistant is an intelligent system designed to enable users to interact with logistics-related documents using natural language queries. The system processes large PDF documents such as logistics manuals and provides accurate answers by combining document retrieval with AI-based text generation.

This system is based on the **Retrieval-Augmented Generation (RAG)** approach, which improves answer accuracy by grounding responses in actual document content rather than relying only on pre-trained knowledge.

1.1 Scope of the Document

This document presents the **Low-Level Design (LLD)** of the Logistics AI Assistant. It includes:

- Detailed explanation of internal modules
- Data flow and processing steps
- Component-level implementation details
- Design decisions and constraints

The document focuses on how the system is implemented at a technical level.

1.2 Intended Audience

This document is intended for:

- Developers responsible for implementation
- Faculty members evaluating the project
- Technical reviewers and examiners
- System architects

1.3 System Overview

The Logistics AI Assistant is an intelligent document-based question-answering system designed to simplify the process of extracting information from logistics-related documents. The system enables users to interact with large and complex documents using natural language queries, eliminating the need for manual searching.

The core of the system is based on the **Retrieval-Augmented Generation (RAG)** architecture. In this approach, the system first retrieves relevant information from the document and then uses a language model to generate accurate and context-aware responses.

Key Features of the System

- Natural language interaction with documents
- Fast and efficient semantic search using FAISS
- Context-aware answer generation using AI
- User-friendly web interface built with Streamlit
- Scalable architecture for handling multiple documents

Key Technologies Used:

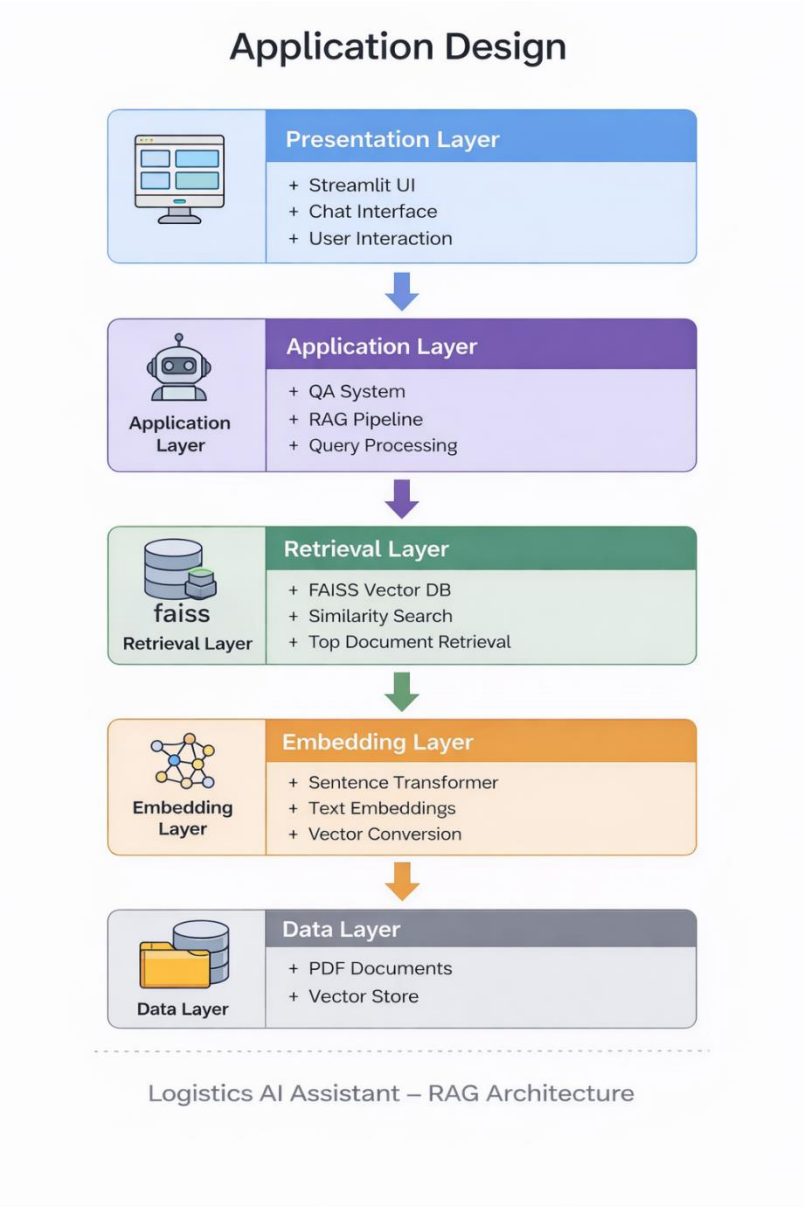
- **Python 3.11** – Core programming language
- **Streamlit** – Frontend UI
- **LangChain** – RAG pipeline orchestration
- **FAISS** – Vector similarity search
- **Sentence Transformers** – Embedding generation
- **Transformers (HuggingFace/OpenAI)** – Language model

2. System Design

The system design describes the internal architecture, workflow, and interaction between different components of the Logistics AI Assistant. The system follows a modular and layered approach to ensure scalability, maintainability, and efficient performance.

2.1 Application Design

The application is designed using a **layered architecture**, where each layer performs a specific function.



Description of Layers:

1. Presentation Layer

- Provides user interface
- Handles navigation and chat interaction
- Displays responses

2. Application Layer

- Controls the flow of data
- Integrates retriever and language model

3. Retrieval Layer

- Performs similarity search using FAISS
- Returns relevant document chunks

4. Embedding Layer

- Converts text into numerical vectors

5. Data Layer

- Stores documents and vector database

2.2 Process Flow

The complete execution flow is as follows:

1. User launches the application
2. Homepage is displayed
3. User clicks "Run Chatbot"
4. Chat interface is activated
5. User enters a query
6. Query is converted into embedding
7. FAISS searches similar document chunks
8. Top relevant chunks are retrieved
9. LLM generates answer using context
10. Answer is displayed to the user

2.3 Information Flow

User Query
↓
Text Embedding
↓
Vector Comparison (FAISS)
↓
Relevant Document Chunks
↓
Context + Query sent to LLM
↓
Generated Answer
↓
Displayed in Chat UI

2.4 Components Design

2.4.1 app.py (User Interface Layer)

Responsibilities:

- Displays homepage and chatbot interface
- Manages navigation using session state
- Handles user input and output
- Applies custom UI styling

Key Features:

- Background image rendering
- Chat interface using st.chat_input()
- Message display using st.chat_message()
- Performance optimization using caching

2.4.2 ingest.py (Document Ingestion)

Responsibilities:

- Load PDF files
- Extract textual data
- Split text into smaller chunks

Why Chunking?

- Improves retrieval accuracy
- Allows efficient embedding

Configuration:

- Chunk Size: 500 characters
- Overlap: 50 characters

2.4.3 embed.py (Embedding Module)

Responsibilities:

- Convert text into vector representations

Model Used:

- sentence-transformers/all-MiniLM-L6-v2

Functionality:

- Captures semantic meaning of text
- Enables similarity comparison

2.4.4 build_index.py (Vector Store Creation)

Responsibilities:

- Generate embeddings for all document chunks
- Store them in FAISS index

Outputs:

- index.faiss → vector index
- index.pkl → metadata

2.4.5 retriever.py (Search Module)

Responsibilities:

- Load FAISS vector database
- Perform similarity search

Functionality:

- Converts query into embedding
- Retrieves top K relevant chunks

2.4.6 qa_system.py (RAG Pipeline)

Responsibilities:

- Integrate retriever with LLM
- Generate final answer

Workflow:

Query → Retriever → Context → LLM → Answer

2.5 Key Design Considerations

1. Performance

- FAISS enables fast similarity search
- Caching reduces model loading time

2. Accuracy

- RAG ensures answers are based on documents
- Reduces hallucination

3. Scalability

- Can support multiple documents
- Can integrate cloud vector databases

4. Usability

- Simple UI with chat interface
- Minimal user effort required

5. Compatibility

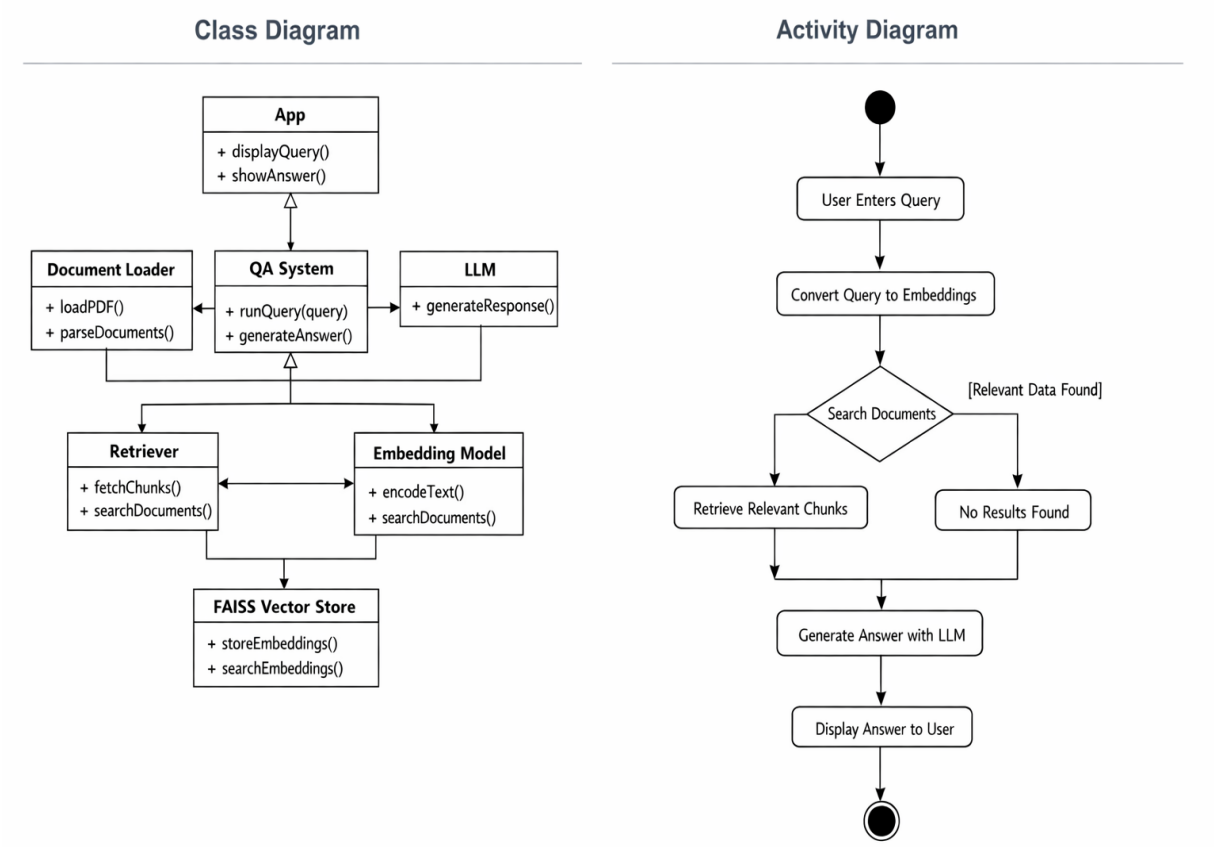
- Python 3.11 ensures stable library support

2.6 API Catalogue

Although no external APIs are used, internal functions act as APIs:

Function	Description
load_documents()	Load and process PDF
split_documents()	Split text into chunks
build_index()	Create FAISS index
load_retriever()	Load retriever
build_qa()	Build QA pipeline
qa.run(query)	Generate answer

3.UML DAIGRAMS



4. References

1. LangChain Official Documentation
2. FAISS Documentation (Facebook AI)
3. HuggingFace Transformers
4. Sentence Transformers Documentation
5. Streamlit Documentation

✅ Conclusion

The Low-Level Design provides a detailed understanding of the internal working of the Logistics AI Assistant.

The system efficiently integrates:

- Document ingestion
- Embedding generation
- Vector-based retrieval
- AI-powered answer generation

This results in a robust and intelligent system capable of transforming traditional document search into an interactive AI-driven experience.